



GraphRAG Inference Hackathon by TigerGraph

Welcome, builders. This page has everything you need to navigate the GraphRAG Inference Hackathon. Bookmark it, share it with your team, and use it as your single source of truth.

About the Hackathon

LLMs are spreading rapidly across every industry, and as they do, **token consumption is exploding**. Companies pay more, wait longer, and hit context limits faster every quarter. This isn't an academic exercise. It's a real, growing pain in production AI today.

The **GraphRAG Inference Hackathon by TigerGraph** is a beginner-friendly, online hackathon where you'll prove that graphs make LLM inference faster, cheaper, and smarter than vector-based RAG alone. You'll build three pipelines side-by-side (raw LLM, Basic RAG, GraphRAG) and let the numbers tell the story.

The headline metric: token reduction with maintained accuracy. Show us how much GraphRAG cuts down without dropping answer quality, and you'll have built the kind of project that gets hiring managers, AI teams, and the open-source community paying attention.

Why Join

Beginner-friendly	No prior TigerGraph or graph experience required. Python and basic LLM API usage is enough.

Open globally	Students, professionals, solo builders, and teams welcome.
\$700 prize pool	Cash prizes plus mentoring from the TigerGraph engineering team for the Top 10.
Make an impact	Your findings and contributions will help solve a real and growing industry-wide problem: token efficiency in production AI.
Free to enter	No fees, no catches.
Free TigerGraph stack access	Full free access to Savanna (\$60 in credits, more if needed), Community Edition, MCP, and the GraphRAG repo.
Direct engineer access	1:1 mentoring sessions, office hours, and WhatsApp support throughout.

Getting Started

You need two things before diving in: a **TigerGraph environment** and the **GraphRAG repo**.

Step 1: Pick Your TigerGraph Environment

Option A: TigerGraph Savanna (recommended). Web-based, zero installation. Sign up at tgcloud.io and get ~\$60 in free credits instantly (more if needed).

Option B: Community Edition. Free, runs locally. Download from dl.tigergraph.com.

Step 2: Clone the GraphRAG Repo

```
git clone https://github.com/tigergraph/graphrag.git
```

This repo is the foundation for Pipeline 3. You'll either use it as-is or build on top of it (more on that in the Problem Statement below).

Step 3: Get an LLM API Key

You'll need access to any LLM provider of your choice for all three pipelines and for the accuracy evaluation. Use whichever you're comfortable with. Most major providers offer free tiers that are more than enough for hackathon-scale usage.

Problem Statement

The Problem

LLMs burn through thousands of tokens to answer complex questions. At scale, that gets expensive fast. **Basic RAG** (vector embeddings + LLM) helps, but vector search retrieves *similar* chunks. It can't *reason* across relationships between entities.

Graphs solve this. A graph organizes data into entities and relationships, performs multi-hop reasoning, and hands the LLM a clean, focused prompt instead of a giant context dump. The result: fewer tokens, faster responses, lower cost, without sacrificing answer quality.

The big bet: GraphRAG beats Basic RAG on every metric that matters. This hackathon is your chance to prove it.

How the Hackathon Works

This hackathon runs in two rounds:

Round 1: Open Hackathon

- Open to all participants globally
- Build the 3-pipeline benchmark (LLM-Only, Basic RAG, GraphRAG)
- Minimum dataset: 2 million tokens of text-based content
- Use any LLM provider. Free options work great (Gemini free tier, Ollama local, etc.)
- Top 10 teams advance to Round 2

Round 2: Production Showcase (Top 10 Only)

- Top 10 teams scale their winning approach to **50–100 million tokens**

- TigerGraph provides **\$50 worth of Gemini API credits per team**, shared via secure endpoints. No out-of-pocket cost, only available during Round 2
 - Final benchmarks featured in the official TigerGraph blog and case studies
 - Winners announced at the end of Round 2
-

What You'll Build

Three pipelines that answer the same questions on the same data, plus a comparison dashboard:

1. **Pipeline 1: LLM-Only.** A prompt goes in, an output comes out. No retrieval. Your worst-case baseline.
2. **Pipeline 2: Basic RAG.** Vector embeddings + LLM. The industry standard today.
3. **Pipeline 3: GraphRAG.** Built on top of the [TigerGraph GraphRAG repo](#). TigerGraph handles entity extraction, relationships, and multi-hop reasoning. The LLM does final synthesis.

💡 **What is TigerGraph GraphRAG?** It's an open-source service that turns your documents into a knowledge graph and exposes APIs for graph-powered Q&A. You'll integrate with it via REST or Python. Think of it as a pre-built backend for the graph layer.

For Pipeline 3, you have **two valid paths** to choose from:

Path A: Use the GraphRAG repo as-is. Deploy the repo via Docker, configure your LLM API key, ingest your dataset, and hit its REST/Python APIs. Treat it like a black box that returns GraphRAG-powered answers. Best for teams that want to focus on benchmarking, the comparison dashboard, and storytelling.

Path B: Customize the GraphRAG repo. Deploy the repo, then go deeper: tune retriever parameters (`top_k` , `num_hops` , `community_level`), swap between retrievers (Hybrid Search, Community, Sibling), modify prompt templates, extend the schema, or add your own logic on top of the existing APIs. Best for teams that want a real engineering challenge and creative freedom.

Both paths are valid. We judge on outcomes (the % improvement and answer accuracy), not on which path you took.

Your **comparison dashboard** is the heart of your project. It's an interactive page where you enter **one query** and it runs that same query through all 3 pipelines at once: LLM-Only, Basic RAG, and GraphRAG. The dashboard then displays each pipeline's response side-by-side, along with these metrics for every pipeline:

- **Tokens used:** prompt + completion tokens per query, the headline metric for cost reduction
- **Response latency:** end-to-end time from question to final answer
- **Cost per query:** calculated from tokens × your LLM provider's pricing
- **Answer accuracy:** measured using two complementary tests: LLM-as-a-Judge (PASS/FAIL grading) and BERTScore (semantic similarity)

This dashboard is what proves your case. One query in, three answers + metrics out, side-by-side. That's the story.

Dataset

You bring your own dataset. Pick any domain: legal, medical, news, support tickets, scientific papers, podcast transcripts, whatever excites you.

The best datasets have a lot of natural connections in them: people, places, events, and topics that link to each other across documents. That's what GraphRAG is built to handle. PDFs, articles, transcripts, and research papers are great choices. Just make sure the content is public domain or properly licensed.

Dataset size requirements:

- **Round 1:** At least 2 million tokens of actual text.
 - **Round 2:** Top 10 teams scale their winning approach to 50–100 million tokens, with \$50 worth of Gemini API credits provided per team for the hackathon.
-

Answer Accuracy

Token reduction alone isn't the win. **Your GraphRAG answers must hold up against your Basic RAG baseline.** That's why every team evaluates answer quality using two complementary Hugging Face approaches:

1. **LLM-as-a-Judge:** A free hosted Hugging Face model grades each answer PASS/FAIL.
2. **BERTScore:** Measures semantic similarity to the correct answer.

 **Note:** GraphRAG requires iterative tuning to ensure high answer accuracy. Teams are expected to adjust key parameters such as chunking strategy, chunk size, hop depth, LLM selection, and prompt design to optimize results. See the [Tuning Guideline](#) in the README for details.

For full setup instructions, code, and bonus point thresholds, check the **Accuracy Evaluation Guide** below:

 [Accuracy Evaluation Guide](#)

How You'll Be Judged

Three things matter. Everything else is a means to these ends.

Criteria	Weight	What We're Looking For
Token Reduction	30%	% improvement in tokens and cost per query vs Basic RAG. Show us the numbers.
Answer Accuracy	30%	Quality maintained or improved. Evaluated with LLM-as-a-Judge + BERTScore.
Performance	20%	Latency, throughput, and overall system efficiency. A fast pipeline matters as much as a cheap one.
Engineering & Storytelling	20%	Clean architecture, working dashboard, clear demo video, blog post that tells the story.

Bonus Points: Strong accuracy is heavily rewarded. Submissions hitting these bars earn extra points in judging:

- High pass rate on the LLM-as-a-Judge evaluation ($\geq 90\%$)
- Strong semantic similarity on BERTScore F1 rescaled (≥ 0.55) or its raw variant (≥ 0.88)

- Hitting both unlocks the maximum bonus

Token reduction with poor answers isn't a win. The bonus rewards teams that prove GraphRAG is genuinely better, not just cheaper.

⚠ Important: Token reduction only counts if your GraphRAG pipeline maintains or improves accuracy compared to your Basic RAG baseline. A team that cuts tokens by 70% but loses 20% accuracy isn't a winner. It's a regression.

Required Deliverables

All submissions go through Unstop directly. Upload everything via your Unstop submission form before the deadline.

1. **Architecture diagram:** clean visual of your system
2. **Comparison dashboard:** side-by-side metrics for all 3 pipelines
3. **Benchmark report:** tokens, cost, latency, and both accuracy metrics per pipeline
4. **Demo video:** 5–7 minute walkthrough
5. **Public GitHub repo:** full source, built on top of the [TigerGraph GraphRAG repo](#)
6. **Blog post:** technical write-up (Medium, Hashnode, [Dev.to](#), or your own)
7. **Social media post:** share on LinkedIn or Twitter, tag @TigerGraph, use #GraphRAGInferenceHackathon
8. **Product Feedback Interview** (*required for Top 5–10 teams*): short interview with the TigerGraph product team about what worked, what didn't, and what would make GraphRAG easier to build with

TL;DR

1. Build 3 pipelines: LLM-only, Basic RAG, and GraphRAG (on top of the [TigerGraph GraphRAG repo](#), use as-is or customize)
2. The hackathon runs in two rounds:
 - **Round 1:** open to all, dataset of at least 2 million tokens of text

- **Round 2:** top 10 teams scale to 50–100 million tokens, with \$50 worth of Gemini API credits provided per team
3. Build an interactive dashboard: one query in, all 3 pipelines run, side-by-side responses + metrics out
 4. Evaluate accuracy with LLM-as-a-Judge + BERTScore
 5. Judging: Token Reduction (30%) + Answer Accuracy (30%) + Performance (20%) + Engineering & Storytelling (20%), with bonus points for high pass rate on the LLM-as-a-Judge evaluation ($\geq 90\%$) and strong semantic similarity on BERTScore F1 rescaled (≥ 0.55) or its raw variant (≥ 0.88)
 6. Ship publicly: GitHub repo, demo video, blog, social post
- Build it. Benchmark it. Prove graph beats tokens.
-

Detailed Guides

Two deep-dive guides to help you build faster and smarter. Click any of them to explore.

Build Faster with MCP

Just a tip: you can connect TigerGraph directly to your AI coding tools (Cursor, VS Code Copilot) using MCP and build with natural language. No GSQL, no boilerplate. Totally optional, but a real time-saver if you're new to TigerGraph.

Accuracy Evaluation Guide

Full Hugging Face setup for evaluating answer quality. Includes the LLM-as-a-Judge + BERTScore approach, code snippets, and bonus point thresholds.

Important Links

Resource	Link
TigerGraph GraphRAG Repo	github.com/tigergraph/graphrag

Resource	Link
TigerGraph MCP	github.com/tigergraph/tigergraph-mcp
TigerGraph Savanna	tgcloud.io
Community Edition	dl.tigergraph.com
TigerGraph Docs	docs.tigergraph.com
Discord Community	discord.gg/4cc7SNqRf
Hackathon WhatsApp Group	chat.whatsapp.com/lwdyhie2gSolR0k2teMtKb
Schedule a 1:1 with Devanshu	calendly.com/devanshu-saxena-tigergraph/20min



Need Help?

Don't get stuck. Reach out:

- **1:1 advice call:** calendly.com/devanshu-saxena-tigergraph/20min
- **WhatsApp:** wa.me/917404313376
- **Hackathon WhatsApp Group:** chat.whatsapp.com/lwdyhie2gSolR0k2teMtKb
- **Discord Community:** discord.com/invite/4cc7SNqRf

We're here to make sure you build something great.